

Project Plan: Self-Service Provisioner Bot

Goal

Allow users to request infrastructure environments (Docker or AWS) through a GitHub issue or a simple web form. Requests are automatically validated against organizational policy, infrastructure-as-code files are generated, and a pull request is opened for review — with actual deployment gated behind manual approval.

Phase 1 — Policy & Governance Layer

- Define what's allowed: environment types, environments, authorized users, default-deny posture.
- Build a standalone PolicyEngine that loads rules from YAML and validates any incoming request (type, env, user).
- Add automation-account bypass so bots/CI users aren't blocked by the human-user allowlist.
- Cover with unit tests before integrating further.

Phase 2 — Infrastructure Template Generation

- Create reusable templates for the two supported environment types (Docker Compose, AWS Terraform).
- Build a generator function that fills in environment-specific values (instance size by environment tier).
- Test generation output for each supported type/environment combination.

Phase 3 — Request Intake & Parsing

- Decide on intake channels: GitHub issues (primary) and a web UI (secondary, for non-technical users).
- Build a lightweight parser that turns free-text issue titles/bodies into structured requests (type, env, user) using keyword detection.
- Expose policy validation and template generation as FastAPI services so they can be called independently or chained.

Phase 4 — Automation Orchestration

- Write the end-to-end handler: on a labeled issue event, parse → validate → generate → branch → commit → PR.

- Add --dry-run mode for safe local testing without touching git/GitHub.
- Add --live mode for the real CI pipeline, with unique per-issue identifiers so generated files always produce a diff.

Phase 5 — GitHub Integration & PR Automation

- Implement git branch creation, committing only generated files, and pushing with a token-based remote.
- Implement PR creation via PyGithub, with duplicate-PR checking and base-branch fallback to the repo default.

Phase 6 — Web UI

- Build a minimal FastAPI front-end so non-technical users can submit requests via a form.
- Form submission creates a labeled GitHub issue, reusing the same automation pipeline (no separate code path).
- Add a dashboard view (open/closed request counts, open PRs) and a results page.

Phase 7 — CI/CD Pipelines

- Standard CI: run the full pytest suite on every push/PR to main across supported Python versions.
- Auto-provision workflow: triggered by the auto-provision label, runs the handler live.
- Apply workflow: triggered on PR merge, runs terraform plan/apply on generated files, gated behind a production environment requiring manual approval — keeping actual infrastructure changes human-reviewed even though the PR creation is automated.

Phase 8 — Documentation & Testing

- README covering local setup, dry-run testing, and the full GitHub Actions flow.
- Issue templates to standardize provisioning requests and bug reports.
- Sample event fixture for local handler testing without a live GitHub event.

PROMPT WE HAVE USED TO CREATE THE PROJECT

AI TOOLS WE USED:

- Claude
- Antigravity
- Github copilot

- Chart GBT

PROMPT USED IN ANTI GRAVITY

AI Usage Note - Key Prompts Used During Development

Prompt 1 - Project Architecture

Design an AI-powered Self-Service Provisioner Bot that allows users to request infrastructure through a web portal.

The system should:

- Accept provisioning requests
- Validate requests using policy rules
- Generate infrastructure templates
- Create GitHub Issues
- Create Pull Requests automatically
- Support Docker, AWS, and Terraform environments
- Track requests through a dashboard

Provide architecture, folder structure, and implementation plan.

Prompt 2 - FastAPI Dashboard

Create a modern FastAPI web application for a Self-Service Provisioner Bot.

Features:

- Dashboard page
- Provision request form
- Environment selection
- Request tracking page
- Responsive Bootstrap 5 UI
- Professional DevOps appearance

Generate complete HTML, CSS, JavaScript, and FastAPI routes.

Prompt 3 - Policy Engine

Create a YAML-based policy validation engine.

Requirements:

- Load policy rules from policy.yaml
- Validate environment types
- Validate infrastructure types
- Return approval or rejection status
- Expose validation through FastAPI APIs

Generate production-ready Python code.

Prompt 4 - AI Request Parsing

Convert natural language infrastructure requests into structured JSON.

Example:

Input:

Create a Docker environment with PostgreSQL and Redis.

Output:

```
{  
  "type": "docker",  
  "services": ["postgres", "redis"]  
}
```

Use Hugging Face Inference API with fallback logic.

Prompt 5 - Infrastructure Generator

Generate infrastructure templates automatically.

Docker:

- docker-compose.yml

AWS:

- Terraform main.tf

Terraform:

- provider.tf
- variables.tf
- main.tf

Store generated files in a generated folder.

Prompt 6 - GitHub Issue Automation

Automatically create GitHub Issues when a request is submitted.

Issue should contain:

- Request ID
- Requester Name
- Environment
- Infrastructure Type
- AI Summary

Apply auto-provision label.

Use PyGithub.

Prompt 7 - Pull Request Automation

Automatically:

- Create Branch
- Commit Generated Files
- Push Changes
- Open Pull Request

Title format:

Provision Environment

Generate complete automation workflow.

Prompt 8 - AI Assistant

Create an embedded AI chatbot for the portal.

Requirements:

- Chat interface
- Welcome message
- Suggested questions
- Provisioning help
- Terraform help
- Docker help
- AWS help

Use Hugging Face models and fallback responses.

Prompt 9 - Premium UI Enhancement

Redesign the UI to look like a modern enterprise DevOps platform.

Add:

- Glassmorphism
- Gradient cards
- Statistics dashboard
- Progress indicators
- Workflow visualization
- Responsive layout
- Dark theme

Generate complete frontend code.

Prompt 10 - Authentication System

Implement authentication using FastAPI.

Requirements:

- Login page
- Logout
- JWT authentication
- Password hashing using bcrypt
- Role-based access

Roles:

Developer

Requester

Admin

Generate database models and APIs.

Prompt 11 - Notification System

Create a notification center.

Notify users when:

- Request submitted
- Request approved
- Request rejected
- Deployment started
- Deployment completed
- Deployment failed

Store notifications in SQLite.

Add notification bell UI.

Prompt 12 - Email Integration

Integrate Resend API.

Send emails:

1. Admin Notification
2. Deployment Started
3. Deployment Successful
4. Deployment Failed

Generate reusable email service.

Prompt 13 - Deployment Automation

Create GitHub Actions workflow.

After PR approval:

Docker:

- Deploy docker-compose.yml

Terraform:

- terraform init
- terraform validate
- terraform plan
- terraform apply -auto-approve

Update deployment status automatically.

Prompt 14 - Deployment Tracking

Create deployment dashboard.

Display:

- Deployment ID
- Environment

- Status
- Start Time
- Completion Time

Statuses:

Pending

Running

Completed

Failed

Provide live status updates.

Prompt 15 - Testing

Generate pytest test cases.

Test:

- Policy validation
- API endpoints
- Request submission
- Request tracking
- GitHub automation
- Notification logic

Provide integration tests and mock tests.

Prompt 16 - README Generation

Generate a complete README containing:

- Project overview
- Architecture
- Setup instructions
- Environment variables

- Running the application
- Testing instructions
- Workflow explanation
- Folder structure
- Screenshots section